

The Index of Operations: representing machine dynamics as hierarchical relational manifolds

Sehaj Singh

Abstract

A machine with one hundred logged variables is usually presented to a learning system as a row of one hundred numbers. That representation is convenient for storage and weak for reasoning because it hides the fact that the variables belong to different physical subsystems, move on different time scales, and constrain one another through interfaces. We introduce the Index of Operations (IOO), a proposed representation in which a machine is a typed hierarchy of operator instances, physical interfaces, and time-evolving relational states. The companion Index of Operators, $\mathcal{I}_{op}$, is a catalogue of reusable component classes such as motors, gearboxes, actuators, and thermal nodes. The Index of Operations, $\mathcal{I}_{ops}$, composes instances of those classes into one system graph and attaches a trajectory to the resulting high-dimensional state. The model input is the typed graph and its state, not a three-dimensional picture of the graph. A low-dimensional projection is used only to inspect the trajectory.

We build the system from scratch, implement physical and statistical edge provenance separately, and test the representation on a controlled coupled motor, transmission, actuator, and thermal system. The first experiment measures whether relational features preserve machine identity under a smooth sensor recalibration. The second replaces a motor variant while retaining its external interface and measures transfer. The manuscript reports the pipeline, the ablations, the limits of simulation, and the cloud reproducibility contract for Kaggle, Google Colab, Modal, and Hugging Face. The central claim is deliberately narrow. IOO is a mechanism for making physical structure explicit to a learning system, not proof that a graph automatically understands physics. The next step is to test the same protocol on public industrial telemetry with machine-level holdout splits, because that is where the representation has to earn its complexity.

1 Introduction

A hardware system does not operate as a table. A motor converts electrical effort into torque, a transmission changes the speed and force available to the next mechanism, an actuator turns that output into motion, and heat accumulates over a slower time scale until the same command no longer produces the same response. The measurements may be stored in one row, but the machine is not one row. It is a set of coupled processes connected through interfaces.

This distinction matters for physical AI because a model trained on flat telemetry can fit the observed data while learning the wrong dependency. It can associate motor current with actuator motion without knowing that the relationship is mediated by a gearbox. It can use ambient temperature as a shortcut for winding temperature. It can fit a particular sensor calibration and then fail when a thermocouple is replaced. A graph helps only if its edges mean something, and an attractive 3D embedding does not by itself create that meaning.

The IOO proposal starts from one load-bearing idea. The representation should preserve the decomposition of a machine into local operators while exposing the interfaces through which those operators become one dynamical system. Each motor, gearbox, actuator, sensor group, or thermal

block has a local state. The complete machine has a global state formed by their composition. The graph is the structural object, the trajectory is the operational object, and the 3D rendering is an inspection surface for a state that still lives in a much larger space.

We define two levels. The Index of Operators, $\mathcal{I}_{\text{op}}$, describes reusable physical classes and their ports. The Index of Operations, $\mathcal{I}_{\text{ops}}$, describes one instantiated machine and how its subsystems are connected. A robot can therefore contain one IOO for each joint, several joint IOOs can form a limb IOO, and the limbs can form the system IOO. This hierarchy is not a visual metaphor. It determines which messages are exchanged, which parameters are shared, and which component can be replaced without flattening the entire system.

The paper makes four contributions. First, it gives a concrete schema for typed operators, interfaces, physical edges, statistical edges, and temporal edges. Second, it implements a fresh end-to-end benchmark without importing code from the other projects in the workspace. Third, it separates the claims that can be tested now, especially recalibration robustness and component substitution, from claims that require real hardware. Fourth, it provides reproducibility adapters for public cloud execution while refusing to treat an unexecuted cloud job as evidence.

2 The IOO representation

2.1 Index of Operators

Definition 1 (Operator class). *An operator class $o \in \mathcal{I}_{\text{op}}$ is a tuple $(\mathcal{P}_o, \mathcal{Q}_o, \mathcal{E}_o, \mathcal{B}_o)$ consisting of ports, physical quantities, equations or constraints, and an admissible operating region.*

A DC motor may expose voltage, current, angular velocity, and torque ports. A gearbox exposes input and output angular velocity and torque. A linear actuator exposes screw speed, extension, and force. A thermal node exposes power, temperature, and heat-flow ports. The catalogue does not claim that a label such as `dc_motor` contains a complete physical theory. It provides a typed prior that tells the model which variables and interfaces are plausible and which parameters can be shared between instances.

2.2 Index of Operations

For one machine, let

$$\mathcal{I}_{\text{ops}} = (V, E_{\text{phys}}, E_{\text{stat}}, E_{\text{temp}}, \mathcal{T}),$$

where V contains component instances and variable nodes, E_{phys} contains declared physical interfaces, E_{stat} contains telemetry-derived couplings, E_{temp} contains temporal adjacency, and $\mathcal{T}$ is the observed trajectory. Every edge carries a relation type and attributes. A correlation edge is therefore not silently promoted into a causal edge.

If operator instance i has local state z_i , the machine state is a point in a constrained product space,

$$z = (z_1, \dots, z_N) \in \prod_i \mathcal{M}_i, \quad \Psi_{ij}(z_i, z_j, \eta_{ij}) = 0,$$

where η_{ij} contains interface parameters. For a transmission with ratio g and actuator screw pitch p ,

$$\omega_{\text{out}} = \omega_{\text{in}}/g, \quad v = \frac{p}{2\pi}\omega_{\text{out}}, \quad F = \frac{2\pi\eta}{p}\tau_{\text{out}}.$$

The global state is therefore not an arbitrary concatenation. It is a product restricted by interfaces, saturation, contact, switching, and thermal boundaries. In machines with impacts or trips the

correct object may be a stratified or hybrid state space rather than one smooth manifold, and the implementation must not hide that distinction.

2.3 More than three axes

A machine with d variables does not require a d -dimensional plot. The graph stores all d variables and their relations. A chart $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^k$ with $k = 2$ or 3 is fitted for inspection and trajectory tracing,

$$z_t = \phi(x_t), \quad t = 1, \dots, T.$$

The projection is not fed back as a replacement for the original state unless an experiment explicitly tests that compression. In this way one can add a hundredth variable without pretending that a three-dimensional display now contains a hundred axes. The variable becomes another node, another interface candidate, and another coordinate in the high-dimensional state while the display remains a projection.

3 Graph construction and learning

Telemetry is standardized with unit-aware preprocessing, timestamp alignment, and missing-value masks. Physical graph edges are supplied by the operator assembly. Statistical edges are estimated within a rolling window by a coupling matrix C_t . For Pearson coupling, edges satisfy

$$e_{ij}^{\text{stat}}(t) \in E_{\text{stat}} \iff |C_{ij}(t)| \geq \delta.$$

Other experiments can replace Pearson coupling with lagged dependence, mutual information, or a learned attention score. These alternatives must remain separate ablations because they answer different questions.

The graph trajectory is

$$\mathcal{G}_{0:T} = \{(G_t, X_t)\}_{t=0}^T,$$

where X_t contains node features and G_t contains typed edges. A heterogeneous message-passing layer has the form

$$h_i^{\ell+1} = U_{\tau_i}^\ell \left(h_i^\ell, \sum_{(j,i,r) \in E_t} M_r^\ell(h_j^\ell, h_i^\ell, e_{ji,t}) \right).$$

The relation index r distinguishes motor-to-gearbox transmission from thermal coupling, containment, and learned statistical dependence. A hierarchical model pools component nodes into subsystem states and subsystem states into a system state. This is where the operator catalogue matters: a new motor instance can share the motor encoder with old instances while retaining its own parameters and interface attributes.

A physical prediction head can be trained with

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_{\text{phys}} \mathcal{L}_{\text{constraint}} + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}.$$

The constraint term is evaluated only for equations whose variables and units are available. Missing physical channels cannot be silently reconstructed and called a verified law.

4 Experiments

4.1 Controlled coupled machine

The fresh benchmark simulates a motor, gearbox, linear actuator, winding temperature, housing temperature, and ambient channel under smooth but changing load. Eight machine identities vary the motor gain and thermal resistance. A second operator variant changes the gearbox ratio and cooling resistance while preserving the external component interface. Each episode contains 800 telemetry steps. The benchmark records command, angular speed, torque, winding temperature, housing temperature, and ambient temperature.

The first task is identity retrieval. A classifier receives one representation from training episodes and predicts the machine identity of held-out episodes. We compare raw channel moments, rank moments, and relational IOO features containing correlation, local loop motion, and temporal change. The second task applies a smooth monotone recalibration to the held-out channels. The relational representation should remain stable under this transformation, while raw magnitude features should move. The third task substitutes the component variant and measures the degradation of the same classifier without retraining.

These experiments are controlled evidence about representation behavior, not evidence of real-machine generalization. They are included because they isolate the mechanism before public industrial datasets introduce missing metadata, inconsistent sampling, and unknown machine identity.

4.2 Cloud reproducibility

The experiment runner is CPU-compatible and writes JSON and CSV artifacts. A Kaggle adapter emits a self-contained kernel. A Colab workflow runs the same module after cloning or uploading the folder. A Modal adapter is provided for independent seeds, and a Hugging Face uploader is explicit rather than automatic. The cloud services are execution surfaces, not sources of scientific authority. Results enter the paper only after the artifact contains the configuration, seed, data manifest, and output checksum.

5 Results and interpretation

The local pipeline produces one row per seed and representation, together with manifold, transfer, and ablation figures. The paper generator is intentionally not allowed to type those numbers into prose. The final numerical table is generated from `metrics.json` after the experiment has run, so a failed or incomplete run remains visibly incomplete.

The key result to test is not whether the IOO feature is always more accurate. A raw model has access to magnitude information that a recalibration-invariant representation intentionally discards. If the machine identities differ mainly by absolute payload or operating level, raw features should win. If they differ by how subsystems couple, relational features should become more competitive. Under sensor recalibration, the expected advantage is stability rather than universal accuracy.

The component substitution experiment tests a second boundary. A known operator family with changed parameters is a realistic transfer case. An entirely new operator class is not zero-shot understood merely because it is placed in a graph. The model must either receive a compatible schema or learn from target data. The manuscript therefore reports known-class substitution, few-shot adaptation, and unseen-class behavior as separate conditions.

6 Limitations

The controlled benchmark is simulated. It does not establish that IOO improves a real factory, robot, motor, or CNC system. Public data connectors are necessary for that test, but a Kaggle account being available does not make a dataset downloaded, cleaned, or leakage-free. The next real-data study should use machine-level splits, preserve timestamps, document units, and compare against temporal baselines rather than only a logistic classifier.

The graph topology can be wrong. A declared physical interface may omit backlash, compliance, or a hidden controller. A statistical edge may reflect a shared workload instead of a mechanism. The representation preserves these distinctions in metadata, but the model can still misuse them. The low-dimensional chart can also distort distances, and curvature or geodesic language should be used only after stability under resampling is demonstrated.

The representation is not invariant to everything. Rank-based relational features resist strictly increasing channel warps, but they lose absolute level information and are sensitive to ties, quantization, record length, and nonstationarity. A production system should carry both relational and calibrated magnitude channels, then measure which faults belong to which axis.

7 Discussion

IOO is best understood as a data structure for physical reasoning. It gives a neural system a place to put the fact that a motor drives a gearbox, that the gearbox drives an actuator, and that heat changes the authority of the motor over time. It also gives an engineer a way to inspect whether the learned coupling agrees with the declared coupling. That is already more useful than presenting one hundred variables as an undifferentiated vector, but it is not the same as saying the AI understands the machine.

The decisive experiment is still ahead. The representation has to leave the controlled simulator, ingest public motor and industrial telemetry, survive machine-level holdout, and then be challenged by a real change in instrumentation or component configuration. If the graph helps only the visualization and not forecasting, fault diagnosis, or transfer, the correct conclusion will be that IOO is an inspection language. If the same typed composition improves data efficiency and transfer under hardware changes, it becomes a candidate interface between engineering systems and physical AI.

That is the stake of the framework. The goal is not to draw the most complicated shape. It is to make the structure that causes the shape explicit enough that a learning system can reuse it when the machine changes.

Data availability

The fresh controlled benchmark is generated deterministically by `src/ioo_experiment.py`. Public dataset adapters and cloud execution instructions are documented in the repository. No private credentials or unexecuted external results are included.

Code availability

The fresh implementation is contained in this repository’s ‘src/’ and ‘cloud/’ directories. The local smoke command is `python -m src.run_experiment --output results/smoke --seeds 0 --machines 4 --episodes 2 --steps 300`.

References

- [1] Y. Bengio, A. Courville and P. Vincent. Representation learning: a review and new perspectives. IEEE TPAMI 35, 1798–1828 (2013).
- [2] M. M. Bronstein et al. Geometric deep learning: grids, groups, graphs, geodesics, and gauges. arXiv:2104.13478 (2021).
- [3] W. Hamilton, Z. Ying and J. Leskovec. Inductive representation learning on large graphs. NeurIPS (2017).
- [4] G. E. Karniadakis et al. Physics-informed machine learning. Nature Reviews Physics 3, 422–440 (2021).
- [5] T. Lyons. Differential equations driven by rough signals. Revista Matemática Iberoamericana 14, 215–310 (1998).
- [6] M. Raissi, P. Perdikaris and G. E. Karniadakis. Physics-informed neural networks. Journal of Computational Physics 378, 686–707 (2019).
- [7] B. Schölkopf et al. Toward causal representation learning. Proceedings of the IEEE 109, 612–634 (2021).
- [8] E. Todorov, T. Erez and Y. Tassa. MuJoCo: a physics engine for model-based control. IROS (2012).
- [9] S. R. S. Varadhan. Diffusion processes in a small time interval. Communications on Pure and Applied Mathematics 20, 659–685 (1967).